



Universidade Federal
de Campina Grande



UNIVERSIDADE FEDERAL DE CAMPINA GRANDE
DEPARTAMENTO DE ENGENHARIA ELÉTRICA E INFORMÁTICA
DISCIPLINA: CAPACITAÇÃO EM PROCESSAMENTO DIGITAL DE
SINAIS

RELATÓRIO PRÁTICO

Relatório 1: Introdução ao Processamento Digital de Sinais

Aluno: Guilherme José Mendes Jerônimo

Professor: Prof. Dr. Edson Porto da Silva

Semestre: 2026.2

Campina Grande - PB

2026

Sumário

1	Resumo	1
2	Introdução	1
3	Desenvolvimento	1
4	Resultados e Análise	2
4.1	Validação do Algoritmo de Convolução	2
4.2	Análise de Desempenho: Python Puro vs. Numba JIT	2
4.3	Processamento em GPU com CuPy	3
5	Conclusão	4

1 Resumo

Este relatório apresenta o estudo e a implementação da operação de convolução discreta em sinais unidimensionais. Avaliou-se o comportamento temporal do algoritmo manual de complexidade $\mathcal{O}(N^2)$ em Python puro, comparando seu desempenho com técnicas de aceleração via compilação JIT (Numba) e computação paralela em GPU (CuPy). Os resultados demonstram o impacto crítico da otimização de laços de repetição em tarefas de Processamento Digital de Sinais (PDS).

O código completo e os experimentos estão disponíveis no [repositório do projeto](#).

2 Introdução

No Processamento Digital de Sinais (PDS), um sinal discreto é representado por uma sequência numérica $x[n]$, onde $n \in \mathbb{Z}$. A operação de convolução entre um sinal de entrada $x[n]$ e a resposta ao impulso $h[n]$ de um sistema Linear e Invariante no Tempo (LIT) é fundamental para determinar a saída $y[n]$ do sistema, definida matematicamente por:

$$y[n] = (x * h)[n] = \sum_{k=-\infty}^{\infty} x[k] \cdot h[n - k] \quad (1)$$

O objetivo prático deste trabalho é compreender a mecânica da convolução discreta, implementar a solução direta via laços de repetição e analisar o gargalo computacional gerado pelo crescimento do tamanho dos sinais (N), avaliando soluções em CPU (Numba) e GPU (CuPy).

3 Desenvolvimento

Para a realização dos experimentos, utilizou-se a linguagem Python com o ecossistema científico formado pelas bibliotecas `numpy` para manipulação de vetores e `matplotlib` para plotagem gráfica.

A implementação manual da convolução consiste em um laço duplo que percorre o comprimento dos sinais de entrada de tamanho N e M , resultando em um sinal de saída de tamanho $L = N + M - 1$. A estrutura do algoritmo envolve:

1. Deslocamento e inversão do sinal $h[n]$;
2. Multiplicação ponto a ponto com $x[k]$;
3. Acúmulo dos produtos no vetor $y[n]$.

Devido aos dois laços `for` aninhados, a complexidade assintótica de tempo do algoritmo manual é $\mathcal{O}(N^2)$ para $N \approx M$. À medida que N cresce, a quantidade de operações aritméticas cresce quadraticamente, tornando o Python puro ineficiente para sinais longos.

4 Resultados e Análise

4.1 Validação do Algoritmo de Convolução

Inicialmente, aplicou-se a convolução em dois sinais sintéticos curtos: um pulso retangular $x[n]$ de amplitude unitária e uma resposta ao impulso $h[n]$ de decaimento exponencial, conforme exibido na Figura 1.

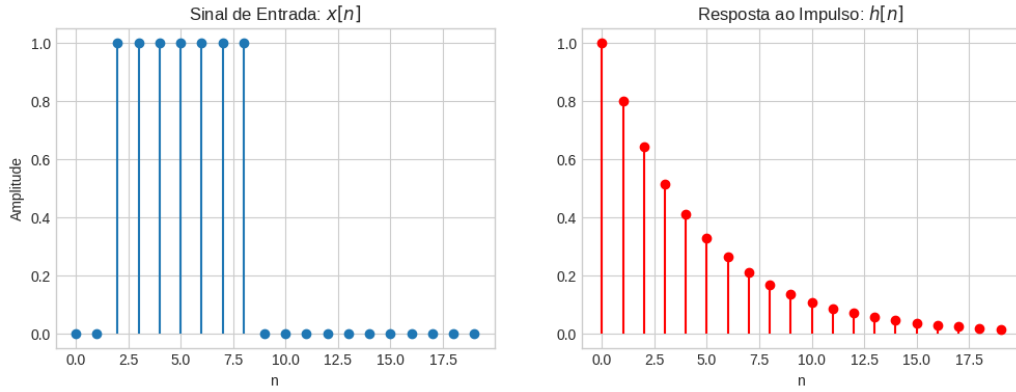


Figura 1: Sinal de entrada $x[n]$ e resposta ao impulso $h[n]$.

O resultado $y[n] = x[n] * h[n]$ (Figura 2) exibe o comportamento esperado de suavização e alisamento do pulso, confirmando a corretude da implementação manual.

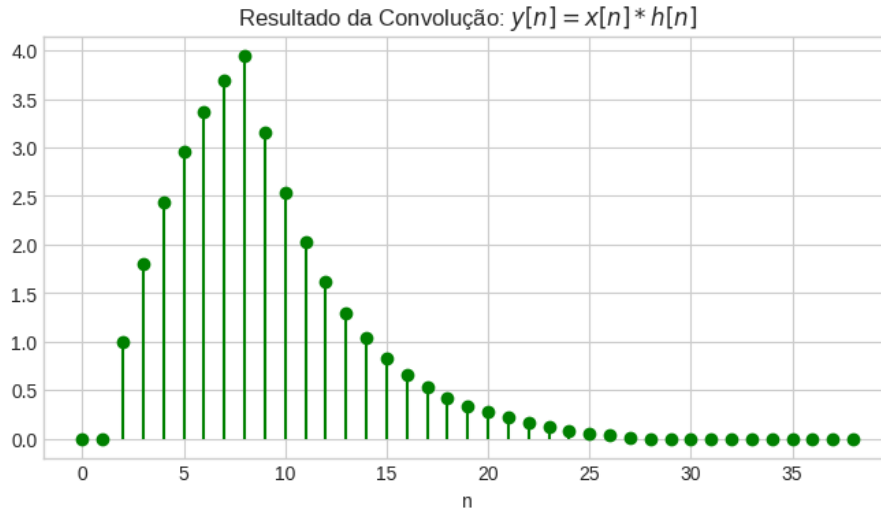


Figura 2: Resultado da convolução discreta $y[n]$.

4.2 Análise de Desempenho: Python Puro vs. Numba JIT

Ao variar o tamanho dos sinais N de 100 até 1600 amostras, o tempo do algoritmo em Python puro apresentou a curva parabólica típica da complexidade $\mathcal{O}(N^2)$, atingindo aproximadamente 0,72 segundos para $N = 1600$ (Figura 3).

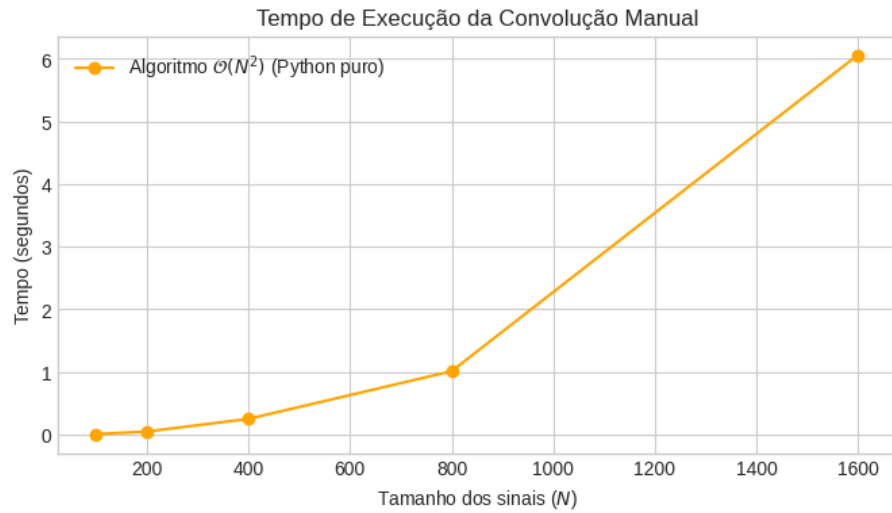


Figura 3: Tempo de execução da convolução manual em Python puro $\mathcal{O}(N^2)$.

Com a introdução do **Numba JIT** (*Just-In-Time Compiler*), o código em laço foi compilado diretamente para linguagem de máquina. Conforme visto na Figura 4, o tempo de execução caiu drasticamente para valores próximos de zero em todas as faixas testadas, eliminando o *overhead* do interpretador Python.

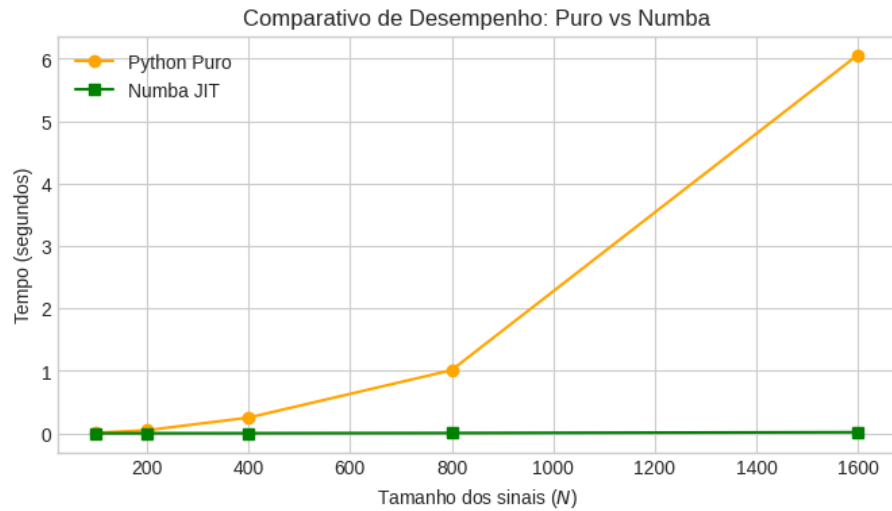


Figura 4: Comparativo de desempenho: Python Puro vs. Numba JIT.

4.3 Processamento em GPU com CuPy

A avaliação do CuPy em ambiente acelerado por GPU (NVIDIA T4 via Google Colab) revelou um comportamento distinto (Figura 5):

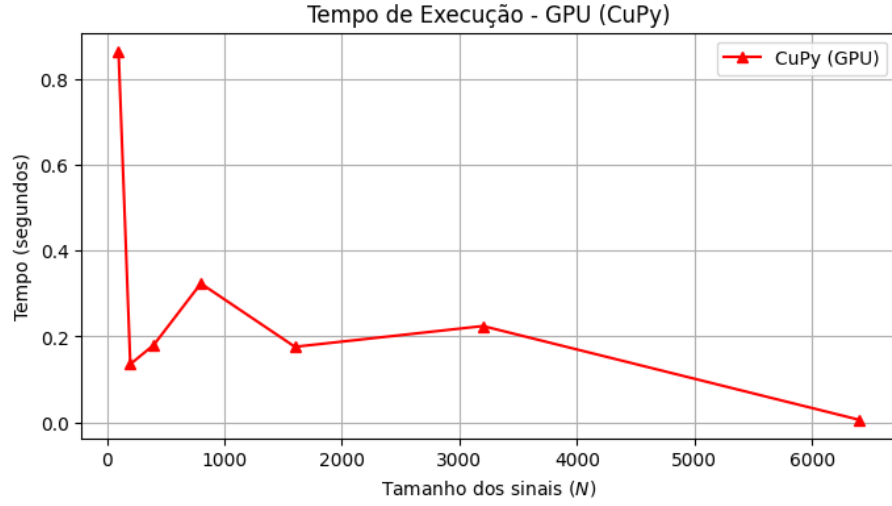


Figura 5: Tempo de execução do CuPy na GPU em função de N .

Observa-se um pico inicial no primeiro ponto ($N = 100$) em torno de 0,85 segundos. Esse fenômeno ocorre devido à inicialização do contexto CUDA, transferência inicial de memória RAM-VRAM e alocação de kernels na GPU (*warm-up*). Após a estabilização, o CuPy processou vetores massivos de até $N = 6400$ amostras em frações de milissegundos, demonstrando a capacidade de paralelização massiva para grandes volumes de dados.

5 Conclusão

O experimento evidenciou que a escrita de laços `for` explícitos em Python puro é inviável para aplicações de PDS em tempo real ou com grande volume de dados devido à complexidade $\mathcal{O}(N^2)$.

Ferramentas de compilação JIT como o Numba oferecem uma solução simples e de altíssimo impacto para acelerar rotinas escritas em CPU sem alterar a estrutura do código. Por outro lado, o uso de GPUs via CuPy e algoritmos baseados em FFT (*Fast Fourier Transform*) mostra-se essencial para processamento de sinais de grande dimensão, apesar do *overhead* inicial de preparação da memória gráfica.